filetype command
Auto detect filetype.

Files
File type associated files are read from runtimepath.
See
    :set runtimepath?

Filetype detection
<PATH>/filetype.vim   common filetype
<PATH>/ftdetect/javascript.vim   per file type detection

#
<PATH>/script.vim   default filetype file
<PATH>/javascript.vim   filetype file

Indent file
<PATH>/indent.vim   default indent file
<PATH>/indent/javascript.vim   custom indent

Plugin load
<PATH>/ftplugin.vim   default plugin
<PATH>/ftplugin/javascript.vim   custom plugin

Commands

```
:help filetype

" Check runtime path
:set runtimepath?

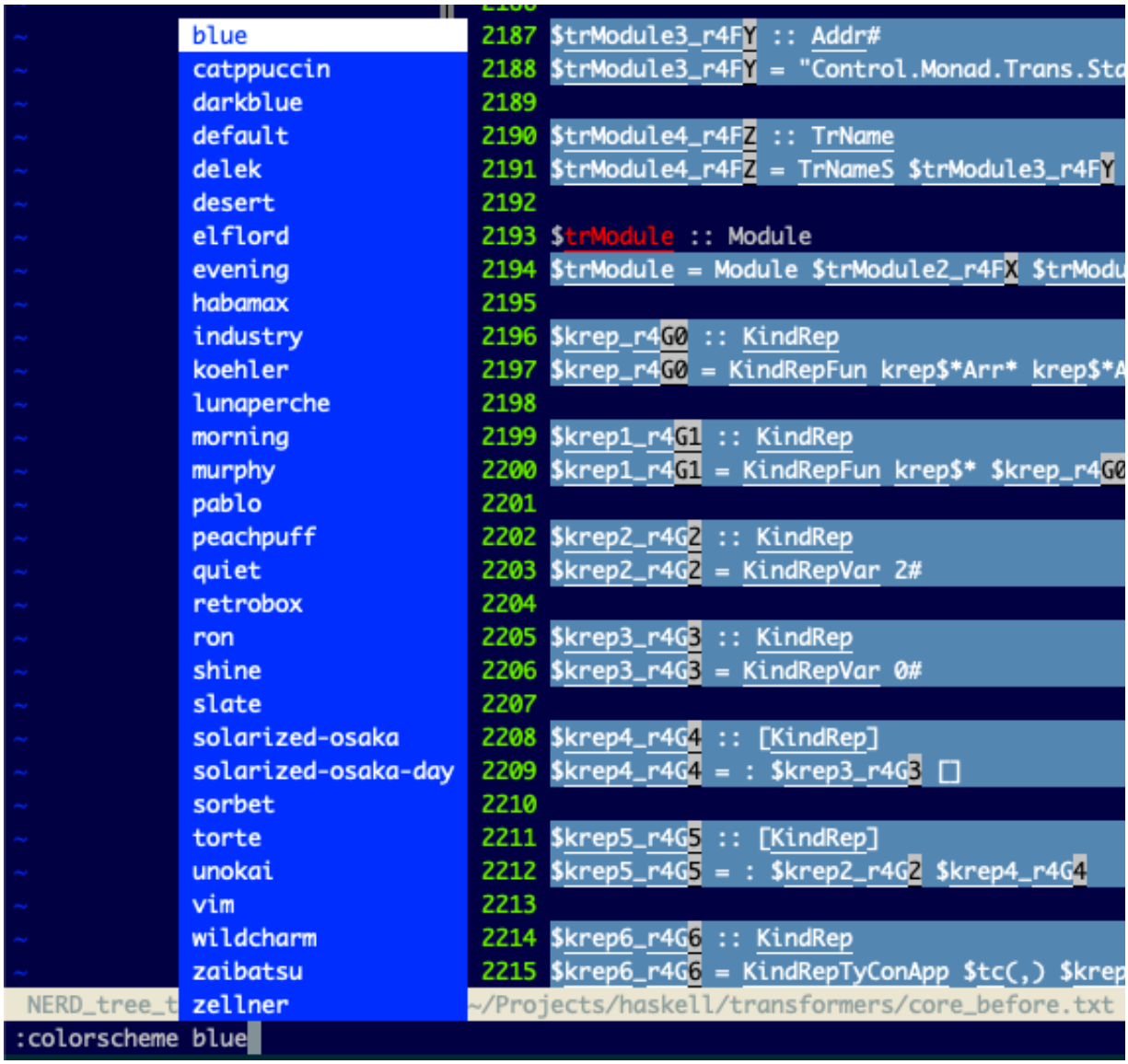

"" set filetype
" Set the filetype option
:set filetype?                     Get current file type
:set filetype=javascript           Manually set filetype

"" setfiletype
" Only used in file type detection files.
" Same as 'set filetype', except sets the filetype only if it has not been set
yet.
:setfiletype javascript


"" :filetype
" Command for managing filetype options
:filetype on                       detection on
:filetype off                      detection off
:filetype plugin on                detection and plugin on
:filetype plugin off               plugin off
:filetype plugin
:filetype
:filetype detect                   redetect


"" colorscheme
" https://stackoverflow.com/questions/7331940/how-to-get-the-list-of-all-
installed-color-schemes-in-vim
:colorscheme                    " get
:colorscheme elflord            " set
```

```
:colorscheme <tab>            " see list of colorschemes
```



Adding a filetype
See :help new-filetype

A new file type may be defined in any path under runtimepath.
e.g. ~/.vim

Method 1: Define in a common filetype.vim
1) Create ~/.vim/filetype.vim

2) Add commands to detect filetype
```
au BufRead *.mytype setfiletype mytype
```

Method 2: Define in separate file under /ftdetect
1) Create directory ~/.vim/ftdetect
2) Create filetype file ~/.vim/ftdect/mytype.vim and add commands to detect type
```
au BufRead *.mytype setfiletype mytype
```

Misc
***) Usage: List all supported file types**
https://vi.stackexchange.com/questions/5780/list-known-filetypes/14990#14990
There is no obvious easy way, but to use an autocompletion hack.

```
" Use commandline completion
```

```
:setfiletype <c-d>

" Use getcompletion command.
:echo getcompletion('', 'filetype')          All filetypes
:echo getcompletion('c', 'filetype')         Filetypes beginning with c
```

==========

Syntax and highlight

Syntax files are located
<runtimepath>/syntax

Syntax and **highlight group name**
Syntactical items are assigned a **syntax group name**.
The syntax group name itself does not specify any color or style.
Color and style are defined a **highlight group**.
A syntax group is assigned a style by **linking** a syntax group to a highlight group.

Default highlight groups are provided
See
:help group-name
e.g.
Comment
Constant
String
...

Commands

```
:help syntax          " Syntax file help
:help 'syntax'        " Syntax option help
:help syn-keyword
:help syn-match
:help syn-region
:help syn-pattern
" See where syntax files are located.
:set runtimepath?
:echo b:current_syntax          Current loaded syntax

"" :syntax
"" :syn
" List defined syntax groups
:syntax
:syn

"" :syntax clear
:syn clear

"" :syntax list {group}
:syn list
:syn               " Same
:syn list MyComment

"" :syntax case [match|ignore]
" Casesensitive or not
:syn case match           case sensitive
:syn case ignore          case insensitive

"" :syntax keyword NAME [{options}] KEYWORD …
" Define syntax group with name NAME for the given keywords
```

```
"
" See :help syn-arguments for options.
:syntax keyword MyTypes int long char         'int', 'long', or 'char' assiged
type MyTypes
:syntax keyword MyTypes contained …           With 'contained' option.

"" :syntax match NAME [OPTS] [excludeenl] PATTERN [OPTS}]
" Define syntax group with name NAME for the given patter
:syn match MyComment /#.*/
:syn match excludenl …

"" :syntax region NAME [matchgroup=GNAME] [OPTS] [keepend] [extend]
[excludenl] start=START [skip=SKIP] end=END
" Matches a region starting with pattern START end ending with pattern END.
" If SKIP pattern is given, does not match the end pattern within the skipped
pattern.
:syntax region MyString start=/"/ end=/"/

"" skip=SKIP
" e.g. Don't want to match escaped double quotes "This is a \"comment\""
:syntax regiou MyString start=/"/ skip=/\\"/ end=/"/

"" keepend, extend
" Default is extend.
" By default, the end pattern does not match the end of contained groups.
" This is necessary for nested regions.
"   e.g. { { } }
:syntax match Braces start=/{/ end=/}/ contains=Braces extend

" Use keepend to match END also against the end of contained groups .
:syntax match InlineComment /\/\/.*$/ contained
:syntax match xPreProc start=/#/ end=/$/ contains=InlineComment keepend
"" matchgroup=GNAME
" Use to different group name for the start/end from the content.
" NAME = name of content
" GNAME = name of start/end
:syntax match BracesContent matchgroup=Braces start=/{/ end=/}/
```

Syntax arguments

```
:help syn-arguments

"" contained
"" contains=GROUP1[,GROUP2…]
"" contains=ALL
"
" contained = group can be contained in another group
" contains = list of other groups it contains.
:syntax keyword MyTodo TODO contained
:syntax match MyComment /#.*/ contains=MyTodo

" Can contain itself.
" e.g. Blocks like { { } }
:syntax region MyBlock start=/{/ end=/}/ contains=MyBlock
:syntax region MyBlock start=/{/ end=/}/ contains=ALL
```

Highlight

```
:help highlight               Highlight command
:help highlight-args
```

```
:help cterm-colors            Color values
:help highlight-groups        List of groups


"" highlight
" List highlight attributes
:highlight               List highlight groups
:highlight ToDo          List ToDo group's attributes


"" highlight clear
" Reset all highlight
:highlight clear              Remove all highlighting of groups added by user
:highlight clear ToD          Clear ToDo group only


"" highlight[!] [default] GROUP {key}={arg} ...
" Set highlight group value
:highlight ToDo cterm=undercurl ctermfg=blue


"" hightlight[!] [default] link GROUP TO
" Create a group alias.
" GROUP is an alias of TO, and hence has the same highlight as TO.
" If TO = NONE, then unlink.
highlight ToDo cterm=undercurl
highlight link ToDo2 ToDo          ToDo2 linked to ToDo
highlight link ToDo2 NONE          Unlink ToDo2
```

Highlight attributes
cterm = color terminal

```
"" term={attr-list}
"
" Terms are:
"   bold
"   underline
"   undercurl
"   reverse
"   inverse
"   italic
term=bold
term=bold,underline          " multiple. Comma-separated, no spaces. Order
does not matter.


"" start, stop
start=
stop=


"" cterm={attr-list}
" Color terminal term
" Same as term
cterm=underline
cterm=underline,bold


"" ctermfg={color-nr}
" Foreground color
ctermfg=0
```

```
"" ctermbg={color-nr}
" Background color
ctermbg=0
```

Color number

```
:help cterm-colors
:help t_co

" usage
ctermbg=NONE            No colors
ctermbg=0               By number
ctermbg=Black           By name
ctermbg=black           Name is case insensitive
```

NR-16 is used if t_CO > 16 (check :set t_CO?).
NR-8 is used otherwise.
For xterm, NR-8 is used (* indicates +8)

NR-16	**NR-8**	**Color name**
0	0	Black
1	4	DarkBlue
2	2	DarkGreen
3	6	DarkCyan
4	1	DarkRed
5	5	DarkMagenta
6	3	DrakYellow
7	7	LightGray, Gray
8	0* (ansi=8)	DarkGray, Gray
9	4* (ansi=12)	Blue, LightBlue
10	2* (ansi=10)	Green, LightGreen
11	6* (ansi=14)	Cyan, LightCyan
12	1* (ansi=9)	Red, LightRed
13	5* (ansi=13)	Magenta, LightMagenta
14	3* (ansi=11)	Yellow, LightYellow
15	7* (ansi=15)	White